

Migrando do GBSwagger para o SwagDoc

Este guia migra uma API escrita com o [Horse](#) do [GBSwagger](#) para o SwagDoc, que escreve a documentação como um documento OpenAPI 3 em vez de um Swagger 2.0.

O que muda

	GBSwagger	SwagDoc para Horse
Especificação	Swagger 2.0	OpenAPI 3 ou Swagger 2.0, pela propriedade <code>SpecVersion</code>
Documentação	Atributos nos controllers e nos models	Objetos do documento, escritos em código
Rotas	Registradas pelos atributos, no registro de paths	Registradas no Horse pela aplicação
Interface do usuário	Swagger UI	Swagger UI ou Scalar
Arquivos da interface	Carregados de um endereço externo	Carregados de um endereço externo ou embutidos

A diferença mais importante é a terceira. No GBSwagger os atributos `SwagGET`, `SwagPOST` e `SwagPath` **também registram as rotas** no Horse, então removê-los remove as rotas da API. A migração tem, portanto, duas partes: registrar as rotas e documentá-las.

1. Dependências

No arquivo `boss.json`, substitua a dependência:

```
- "github.com/gabrielbaltazar/gbswagger": "^3.0.7",  
+ "github.com/marcelojaloto/SwagDoc": "^2.0.0",
```

No search path do projeto, substitua as três pastas do GBSwagger pelas duas do SwagDoc:

```
..\..\modules\gbswagger\Source\Horse;..\..\modules\gbswagger\Source\Core;..\..\modules\gbswagger\Source\Validator;  
+..\..\modules\SwagDoc\Source;..\..\modules\SwagDoc\Integrations\Horse\Source;
```

2. O middleware

```
- Horse.GBSwagger,  
+ Horse.SwagDoc,
```

```

- THorse.Use(HorseSwagger('api/help'));
+ SwagDocConfig.UserInterfaceRoute := '/api/help';
+ SwagDocConfig.DocumentRoute := '/api/help/openapi.json';
+ SwagDocConfig.DiscoverRoutes := False;
+ THorse.Use(HorseSwagDoc);

```

O `DiscoverRoutes` escreve as rotas registradas no Horse que ainda não foram documentadas. É útil enquanto a migração não termina, para enxergar o que falta, e é desligado quando todas as operações estão documentadas. Lembre-se de que a descoberta compara o caminho da rota com o caminho do documento: quando o documento declara um server com um caminho base, como `http://localhost:9000/api`, os caminhos do documento não repetem o prefixo `/api` e todas as rotas parecem não documentadas para a descoberta.

3. As rotas

Os atributos do GBSwagger registram as rotas, então agora é a aplicação que as registra. Um controller cujos métodos respondem às rotas continua funcionando com um método de classe que o cria, chama o método e o destrói:

```

class procedure TController.Execute(Request: THorseRequest; Response: THorseResponse;
  const Action: TControllerAction);
var
  Controller: TController;
begin
  Controller := Self.Create(Request, Response);
  try
    Action(Controller);
  finally
    Controller.Free;
  end;
end;

```

O construtor do controller precisa ser `virtual`, para que `Self.Create` crie a classe da rota. Com ele, cada rota é registrada assim:

```

THorse.Get('/api/tasks',
  procedure(Request: THorseRequest; Response: THorseResponse)
  begin
    TTaskController.Execute(Request, Response,
      procedure(Controller: TController)
      begin
        TTaskController(Controller).List;
      end);
  end);

```

A seção `initialization` que registrava o controller no GBSwagger é removida:

```

-initialization
- THorseGBSwaggerRegister.RegisterPath(TTaskController);

```

4. A documentação das operações

Cada atributo vira uma propriedade dos objetos do SwaggerDoc. O método `Route` traduz uma rota escrita com a sintaxe do Horse e acrescenta um parâmetro para cada variável da rota.

GBSwagger	SwaggerDoc
[SwaggerPath('tasks', 'Tasks')]	SwaggerDocApi.Route('/tasks') e Operation.Tags.Add('Tasks')
[SwaggerGET('List of all tasks')]	Path.AddOperation(ohvGet) e Operation.Summary
[SwaggerPOST('Create a new task')]	Path.AddOperation(ohvPost)
[SwaggerPUT('{id}', 'Change data')]	SwaggerDocApi.Route('/tasks/{id}').AddOperation(ohvPut)
[SwaggerPATCH(...)] , [SwaggerDELETE(...)]	ohvPatch , ohvDelete
[SwaggerPathParam('id', 'Task Id')]	Escrito pelo <code>Route</code> ; a descrição é definida em <code>Path.Parameters[0]</code>
[SwaggerParamQuery('count', 'Description', False, False)]	TSwagRequestParameter com <code>InLocation := rpiQuery</code>
[SwaggerParamBody('Task data', TTaskModel)]	Operation.RequestBody.AddMediaType('application/json').Schema.Name := 'task'
[SwaggerResponse(200, TTaskModel, 'Task list')]	TSwagResponse com <code>StatusCode</code> , <code>Description</code> e o media type
[SwaggerResponse(404)]	TSwagResponse com a descrição do status

Uma operação documentada com o SwagDoc:

```
vOperation := SwagDocApi.Route('/tasks/{id}').AddOperation(ohvGet);
vOperation.OperationId := 'getTask';
vOperation.Summary := 'Get data for a specific task';
vOperation.Tags.Add('Tasks');

vResponse := TSwagResponse.Create;
vResponse.StatusCode := '200';
vResponse.Description := 'Task data';
vResponse.AddMediaType('application/json').Schema.Name := 'task';
vOperation.Responses.Add(vResponse.StatusCode, vResponse);
```

Operações escritas em código são mais longas que atributos e, em troca, alcançam todo o modelo da especificação: request bodies com vários media types, exemplos, links, callbacks, webhooks e os objetos reutilizáveis dos components.

5. Os schemas

Os atributos dos models são substituídos por schemas escritos com o `TJsonSchema` e acrescentados ao documento como definitions. O SwagDoc os escreve sob `definitions` em um documento Swagger 2.0 e sob `components/schemas` em um OpenAPI 3.

GBSwagger	SwagDoc
[SwagProp('title', 'Description', True, False)]	AddField<string>('title', 'Description') com Required
[SwagString(100)]	TJsonFieldString com MaxLength
[SwagString(36, 36)]	TJsonFieldString com MinLength e MaxLength
[SwagNumber(0, 4)]	TJsonFieldInteger com MinValue e MaxValue
Uma propriedade de um tipo classe	AddField<T> do tipo, ou um \$ref para outro schema

```
vSchema := TJsonSchema.Create;
vSchema.Root.Description := 'A task of the manager';

vTitle := TJsonFieldString(vSchema.AddField<string>('title', 'Task title description.'));
vTitle.Required := True;
vTitle.MaxLength := 100;

vDefinition := TSwagDefinition.Create;
vDefinition.Name := 'task';
vDefinition.JsonSchema := vSchema.ToJson;
SwagDocApi.Definitions.Add(vDefinition);
```

Um schema escrito à mão referencia outro com `#/components/schemas/<nome>` em um documento OpenAPI 3 e com `#/definitions/<nome>` em um Swagger 2.0. Aplicações que publicam as duas versões escolhem o prefixo pela propriedade `SpecVersion` do documento.

6. A segurança

```
- Swagger
- .AddBearerSecurity
- .AddCallback(HorseJWT(TJWTSettings.SECRET_KEY))
```

O middleware do token é acrescentado pela aplicação, nas rotas que ele protege:

```
THorse.Use('/api/tasks', HorseJWT(TJWTSettings.SECRET_KEY));
```

E o documento declara o esquema e as operações que o exigem:

```
vBearer := TSwagSecurityDefinitionHttp.Create;
vBearer.SchemeName := 'bearerAuth';
vBearer.Scheme := 'bearer';
vBearer.BearerFormat := 'JWT';
SwagDocApi.SecurityDefinitions.Add(vBearer);

vRequirement := vOperation.AddSecurityRequirement;
vRequirement.AddScheme('bearerAuth', []);
```

7. As informações da API

```
- Swagger
- .Info
- .Title('Tasks API')
- .Description('The API aims to manage information about tasks.')
- .Contact
- .Name('Marcelo Jaloto')
- .&End
- .&End
- .BasePath('api')
- .AddProtocol(TGBSwaggerProtocol.gbHttp)
+ SwagDocApi.Info.Title := 'Tasks API';
+ SwagDocApi.Info.Description := 'The API aims to manage information about tasks.';
+ SwagDocApi.Info.Contact.Name := 'Marcelo Jaloto';
+ SwagDocApi.AddServer('http://localhost:9000/api', 'Local server');
```

No OpenAPI 3 o host, o caminho base e o protocolo são substituídos pelos servers. Uma aplicação que continua publicando um documento Swagger 2.0 usa as propriedades `Host`, `BasePath` e `Schemes`, e o SwagDoc as traduz para um server quando o documento é escrito como OpenAPI 3.

8. A versão da especificação

O documento publicado pelo middleware é escrito como OpenAPI 3. Uma API que precisa manter o documento Swagger 2.0 durante a migração muda uma propriedade:

```
SwagDocApi.SpecVersion := svSwagger2;  
SwagDocConfig.DocumentRoute := '/api/help/swagger.json';
```

Os mesmos objetos produzem os dois documentos, então as duas versões podem ser comparadas antes de a antiga ser abandonada.

Checklist

- ☐ A dependência e o search path apontam para o SwagDoc.
- ☐ Todas as rotas que os atributos registravam estão registradas no Horse.
- ☐ Todas as operações estão documentadas, e o `DiscoverRoutes` não mostra mais nada.
- ☐ Os caminhos do documento correspondem às rotas, considerando o caminho base do server.
- ☐ Os models não usam mais os atributos do GBSwagger.
- ☐ O middleware do token protege as rotas que exigiam a segurança.
- ☐ O documento é válido nas duas versões, se a API publica as duas.

Um exemplo completo

O exemplo [tasks-manager-horse](#) foi migrado com estes passos: sete operações, cinco schemas, autenticação com JWT e um documento OpenAPI 3.2.1.